

Assignment 5 — Solution Key

Addressing Modes, CPU Organization, and Bus Structures

PCC CS-401: Computer Organization and Architecture

August 7, 2026

5.1 Conceptual

Q1. Direct addressing encodes the effective address A directly in the instruction's address field, and that field is fixed at *compile time* — it cannot change while the program runs. But $A[i]$'s address changes on every loop iteration as i changes. **Displacement (indexed) addressing** should be used instead: $EA = [R_{\text{base}}] + [R_{\text{index}}]$, where R_{index} holds i . Because EA is recomputed from a *register* on every execution of the instruction, the same instruction correctly accesses a different array element on every iteration — exactly the property direct addressing lacks.

Q2. Indirect addressing: the instruction's address field A points to a memory location, and *that* memory location holds the actual operand's address, so $EA = M[A]$. This costs two memory reads: one to fetch the pointer from $M[A]$, one to fetch the operand from $M[EA]$. Register-indirect addressing: a register itself holds the operand's address, so $EA = [R2]$ (for example), costing only one memory read (to fetch the operand) plus one register read (to get the address, which is free/fast since it needs no bus transaction). **Register-indirect is cheaper** because the pointer lives in a register, not memory, eliminating one of indirect addressing's two memory accesses.

5.2 Numerical

Q3. Displacement addressing: $EA = [R3] + d = 2048 + 30 = 2078$. **Cost:** one register read (to obtain $[R3]$) plus one memory read (to fetch the operand from $M[2078]$) — two accesses total, one of which is a (cheap) register read.

Q4. 3 control steps, identical to $R1 \leftarrow R2 + R3$. The single-bus datapath's control-step count depends only on the *shape* of the operation (a two-operand ALU op needs to stage one operand in Y , compute and stage the result in Z , then write back), not on which specific registers are named:

$$\begin{aligned}T_1 : Y &\leftarrow [R5] \\T_2 : Z &\leftarrow [Y] + [R6] \\T_3 : R4 &\leftarrow [Z]\end{aligned}$$

5.3 Design

Q5. Subtraction has the identical three-step shape as addition; only the ALU's selected function changes, at T_2 :

$$\begin{aligned}T_1 : Y &\leftarrow [R2] && (\text{R2 onto bus, latched into Y}) \\T_2 : Z &\leftarrow [Y] - [R3] && (\text{R3 onto bus} \rightarrow \text{ALU; ALU subtracts; result} \rightarrow Z) \\T_3 : R1 &\leftarrow [Z] && (Z \text{ onto bus, latched into R1})\end{aligned}$$

3 control steps, same as addition, because the bottleneck forcing three steps is structural (the bus can carry only one operand per cycle, and the ALU needs two, then a write-back) — it does not depend on which ALU operation is selected, only on the fact that it is a two-operand, one-result operation.

Q6. On the three-bus datapath, both operands are read and the result is written back in a single cycle:

$$T_1 : R7 \leftarrow [R4] + [R5], \quad A = [R4], \quad B = [R5], \quad C = \text{result}.$$

Bus A carries $[R4]$ into the ALU's first input; Bus B carries $[R5]$ into the ALU's second input; Bus C carries the ALU's output back into R7 via the destination decoder. This instruction requires **1 control step**, since Bus A and Bus B are separate physical paths that can each drive a different register's contents in the same cycle, and Bus C is not competing with either for the write-back.